Merge Sort in C++

CSD2183 Data Structures — Week 02 Lab — Exercise 3

Objective

A merge compares the two unprocessed heads of its sorted sources and writes the smaller one; its main loop stops as soon as one source runs out of keys, and a cleanup loop then copies the remainder. You will now implement that idea in C++. What differs between implementations is not the comparisons but where the temporary data lives.

One scheme gives each `merge()` call two fresh arrays, `left` and `right`, copies the two halves into them, and merges back into `ar`. Your version instead reuses one auxiliary vector, `scratch`, for the whole sort. During `merge(ar, scratch, lo, md, hi)`:

1. the two sorted source ranges stay where they are: the **left range** `ar[lo .. md]` and the **right range** `ar[md + 1 .. hi]`;
2. their merged result is written to `scratch[lo .. hi]`; and
3. that merged range is copied back to `ar[lo .. hi]`.

The comparison logic is the same in both schemes. The index bookkeeping is not, so pseudocode written for two fresh arrays will not transfer line by line.

> **i** Why one reusable vector?
>
> Both implementations need $\Theta(n)$ peak auxiliary space. Reusing one vector avoids repeatedly allocating temporary vectors during the sort; the same workspace is passed through all recursive calls.

Timing and Rules

- You have **50 minutes** in total, including the time to read this document, work through the first steps below, and submit to Gradescope.
- **Test your code locally before you go to Gradescope.** The Gradescope link is one-way: the Examena browser gives you no Back button, so once you follow it you cannot return to xSITe. Build your code and run the supplied tests in your own terminal first, and follow the link only once your program compiles and sorts correctly.
- **Once you are on Gradescope, resubmit as often as you like.** Gradescope takes unlimited resubmissions and only your last one counts, so submit as soon as your code passes the local tests—do not save submitting for the end. Keep the last **5 minutes** free to confirm that the submission on Gradescope is the one you want.
- This exercise is **not graded this week**; use the Gradescope feedback to check your implementation.
- Work individually. Ask an instructor if something is unclear or goes wrong.

- All local applications and offline files are allowed.
- **Do not use generative AI or AI-assisted coding tools.**

First Steps, Once the Session Has Started

You are reading this from inside the extracted archive, so the download is behind you. Work through these before you write any code:

1. Check that the extracted project sits in **its own folder**, not loose among your other downloads. Move it now if it does not.
2. Open the project in your editor, and keep this PDF open beside it.
3. In a terminal opened in the project folder, run:

```
make clean
make
make run FILE=test_reverse.txt
```

The untouched starter should build. It emits three expected `unused variable` warnings for `i`, `j`, and `k`, then prints the test input unchanged, because the algorithm is not yet implemented.

> ⚠ If anything goes wrong, tell your lab instructor immediately
>
> That covers a `make` that does not work, a Gradescope page you cannot reach, a laptop that fails, and any other technical problem that stops you working normally. **Ask before you start repairing anything yourself.** Your instructor may already know the fix, and can tell you whether a repair is worth the time you have left; restarting Examena or trying an improvised workaround on your own can cost you more time than it saves.
> Examena has been breaking builds on some WSL setups even when every application is permitted, which is why this exercise is not graded this week.

Your Task

Open `merge_sort.cpp`. This is the **only source file you should edit** and the only file you submit to Gradescope.

Do not modify `merge_sort.h`. The autograder uses it to test `merge()` and `mergeSortRange()` separately, so its feedback can identify which part is failing.

Complete the five marked `TODO` sections.

Part 1 — Complete `merge()`

The function

```
merge(ar, scratch, lo, md, hi)
```

has this contract:

- **Precondition:** `ar[lo .. md]` and `ar[md + 1 .. hi]` are each sorted in ascending order.
- **Postcondition:** `ar[lo .. hi]` contains the same keys, sorted in ascending order; positions of `ar` outside this range are unchanged; and `scratch[lo .. hi]` holds that same merged sequence.

! `scratch` is the workspace

The last clause is part of the contract and the autograder checks it. The merge must be performed **through `scratch`**: allocating temporaries of your own, merging into those, and leaving `scratch` untouched does not satisfy it, even if `ar` comes out sorted.
Three rules follow. The autograder checks each one, and each affects your score:

- **`merge()` and `mergeSortRange()` create no workspace of their own** — no local vector, array or buffer, and nothing from `new` or `malloc`. Only `mergeSort()` allocates, and only once. That is the whole point of the exercise.
- **Write the merge yourself.** `std::sort`, `std::stable_sort`, `std::merge`, `std::inplace_merge` and `qsort` are not permitted for this task.
- **`mergeSortRange()` recurses**, as Part 2 describes.

The starter already declares

Index	Meaning
`i`	next destination position in `scratch`
`j`	next unprocessed key in `ar[lo .. md]`
`k`	next unprocessed key in `ar[md + 1 .. hi]`

Useful mental model

While the merge runs, keep this invariant in mind: `scratch[lo .. i - 1]` already holds the smallest keys of the two source ranges taken together, in ascending order.

Write the four remaining steps, marked `TODO 1` to `TODO 4` in the file:

1. **Main merge loop.** While both source ranges still contain an unprocessed key, compare `ar[j]` with `ar[k]`, write the smaller key to `scratch[i]`, and advance the appropriate source index and `i`. When the two keys are equal, take the one from the left range—that is, compare with `<=`.
2. **Left cleanup loop.** Copy any unprocessed keys remaining in the left range to `scratch`.
3. **Right cleanup loop.** Copy any unprocessed keys remaining in the right range to `scratch`.
4. **Copy back.** Copy `scratch[lo .. hi]` into `ar[lo .. hi]`.

Exactly one cleanup loop copies keys in any one call, but both are required because either source range can empty first.

Part 2 — Complete `mergeSortRange()`

Fill in `TODO 5`, the body of

```
mergeSortRange(ar, scratch, lo, hi)
```

so that it:

1. returns immediately when the range contains at most one key;
2. computes the midpoint as `lo + (hi - lo) / 2`;
3. recursively sorts `ar[lo .. md]`;
4. recursively sorts `ar[md + 1 .. hi]`; and
5. calls `merge()` on the two sorted ranges.

Do not assume that the number of keys is a power of 2. The implementation must handle odd-sized ranges as well.

> **!** Important
>
> `lo`, `md`, and `hi` have type `std::size_t`, which is unsigned. Do not write conditions that rely on an index becoming negative. Check the base case directly before making recursive calls.

Compile and Test Locally

After editing, run `make`, then test several supplied cases:

```
make
make run FILE=test_sorted.txt
make run FILE=test_reverse.txt
make run FILE=test_duplicates.txt
make run FILE=test_odd.txt
```

A bare file name is looked up in `tests/`; an existing path is used as given. The public tests include already sorted input, reverse order, duplicates, negative keys, and an odd number of keys. The autograder also checks boundary cases and much larger arrays, including a reverse-sorted input of 200,000 keys under a time limit. Merge sort is $\Theta(n \lg n)$ and finishes that comfortably; a merge that makes room by shifting keys within `ar`, instead of writing them through `scratch`, is quadratic and will run out of time.

For example, input

```
7, 3, 9, 2, 5
```

should produce

```
2, 3, 5, 7, 9
```

Do not rely on one successful test. Think about what your code should do for an empty input, one key, equal keys, and larger inputs as well.

Submit to Gradescope

The Examena-controlled xSITe item uses a **Written Response** question, but your code goes to **Gradescope only**.

1. When your program builds and passes the tests above, open Gradescope through the link inside the xSITe item. The link is one-way: once you follow it you cannot get back to xSITe. You will not need to—everything from here on happens in your editor and on Gradescope.
2. Upload **only** `merge_sort.cpp`.
3. Read the autograder feedback, fix your code locally, and resubmit if necessary.
4. Before ending Examena, confirm that Gradescope shows the submission you want to keep.

You do **not** need to enter anything in the xSITe Written Response text box.

> **!** Important
>
> Submit to Gradescope **before** you end the Examena session. The most recent submission is the one that counts.

Finished Early?

First make sure your working `merge_sort.cpp` has been submitted successfully. Then copy the **entire project folder**, and in the copy add a counter that records how often the main merge loop compares a key from the left range with a key from the right range. Print the count to `std::cerr`, so the normal sorted output on `stdout` is unchanged.

Run that counting version on these two eight-key inputs:

```
20, 17, 12, 9, 8, 5, 3, 1
```

```
12, 5, 20, 8, 3, 17, 1, 9
```

Both are arrangements of the same eight distinct keys, so both cost the same number of copies. Predict which one needs fewer left–right comparisons, then check.

You counted these two comparison totals earlier today, for an implementation that stored its temporary data differently. Do your numbers agree with those? If not, work out where your counting differs from the earlier definition.

> **⚠** Warning
>
> Do not overwrite a working Gradescope submission with experimental code. The `makefile` lists the project source files explicitly, so a backup `.cpp` file will not be compiled accidentally; nevertheless, keeping experimental work in a copied folder is the safest workflow.